Ônibus SP × Chuva — relatório de 24 e 25/10/2026

Silver completa (velocidade por trecho e chuva), pipeline diário, dashboard e próximos passos

Gabriel Graciano Dias · IME-USP / MAE0350
25/10/2026

Resumo

Em dois dias, a camada **silver** do projeto ficou completa e automatizada. Hoje existe, no Databricks, tudo o que a análise precisa para responder à pergunta do projeto — *onde a chuva mais degrada a velocidade dos ônibus de São Paulo* — faltando só a camada **gold**, que cruza velocidade e chuva.

O que	Estado em 25/10
Velocidade por trecho entre paradas, a cada 10 min	✅ 61 dias (25/08 → 24/10), 280 M linhas, 29.946 trechos físicos, 1.309 linhas de ônibus
Chuva em grade de 10 min, 141 postos	✅ 01/08 → 25/10; 95 postos confiáveis cobrindo a cidade
Pipeline diário (Drive → bronze → silver)	✅ 6 etapas, todo dia às 10h, aviso de falha por e-mail
Dashboard (AI/BI)	✅ duas abas: a feita à mão e a gerada por API
Documentação	✅ notebooks comentados, descrições de tabelas e colunas, `docs/` e `CLAUDE.md` no GitHub
Gold (efeito da chuva, KRs)	⬜ próximo passo

Resultado já visível: a velocidade comercial média dos ônibus é **15,5 km/h** (dia útil 15,1; sábado 16,5; domingo 17,9). Em dia útil, ela vai de ~23 km/h de madrugada a **11,9 km/h às 18h**, o horário mais lento do dia. Em dias úteis, a velocidade média cai conforme chove mais (14,4 km/h nos dias secos → 13,6 nos chuvosos) — uma primeira olhada, que a gold vai medir direito.

Três correções importantes nestes dois dias, todas pegas por checagens:

1. A projeção das paradas no traçado errava em linhas de ida e volta pela mesma avenida (23 mil trechos de 0 m) — corrigida em três versões.
2. A queda da velocidade de madrugada era artefato: **frota estacionada fora do terminal**. Nova regra `parado_longo`.
3. O horário do SIBH é **UTC**, não horário de Brasília (o `CLAUDE.md` estava errado).

1. Onde estávamos

No início de 24/10, a silver tinha GTFS, posições limpas e posições projetadas no traçado (`posicao_rota`), mas **parava em 11/10** e ficava fora do pipeline diário. Faltavam as duas etapas que transformam posições em velocidade e toda a chuva — a extração do SIBH estava em 4 de 17 blocos.

O fluxo de dados do projeto, com o que foi feito nestes dois dias em negrito:

```
API Olho Vivo —1 min—▶ VM —2h—▶ Google Drive —10h—▶ Databricks
                                                      |
 bronze.posicao —▶ silver.posicao_limpa —▶ silver.posicao_rota
                                                      |
            —▶ **silver.deslocamento** —▶ **silver.velocidade_trecho** ◀— **silver.gtfs_trechos**
                                                      |
 SIBH (manual) —▶ **bronze.chuva_sibh** —▶ **silver.chuva_10min** + **silver.postos**
                                                      |
                                              gold (próximo passo)
```

2. De posições para velocidade

2.1 `silver.deslocamento` — pares de posições consecutivas

O que é. Cada linha é um **par de posições seguidas do mesmo ônibus no mesmo dia**: de onde para onde ele foi na régua da rota (metro ao longo do traçado) e em quanto tempo. É a unidade de observação da velocidade: a 1 minuto entre posições, um ônibus anda 200–700 m — mais que um trecho entre paradas (~300 m).

Decisão de desenho: classificar, não apagar. Todo par fica na tabela com uma `situacao`; só os `ok` seguem para a velocidade. Isso deixa a checagem simples e permite testar depois o efeito de cada regra.

`situacao`	Regra	Por quê	% dos pares
`buraco`	mais de 5 min entre as posições	coleta perdida; interpolar seria inventar	0,5%
`nova_viagem`	mudou de traçado, ou foi do fim para o início da rota	uma viagem terminou e outra começou	2,3%
`terminal`	as duas pontas a até 200 m do início ou do fim **e** andou menos de 50 m	ônibus esperando no terminal	23,4%
`recuo`	andou mais de 30 m para trás	inclui o "sentido atrasado" da API	1,7%
`rapido`	mais de 80 km/h	impossível para ônibus urbano	0,3%
`parado_longo`	parado ≥ 20 min seguidos (ver 2.4)	frota estacionada fora do terminal	3,8%
`ok`	o resto	vai para a velocidade	**68,1%**

Ônibus parado ou lento no meio da rota fica `ok`. Velocidade perto de zero num engarrafamento é justamente o sinal que queremos comparar com e sem chuva. (O artigo de Niterói corta tudo abaixo de 2 km/h — aqui não, de propósito.)

Ecos saem antes. A API às vezes reenvia uma posição antiga com horário novo (~1% das posições). Eco = mesmas coordenadas de 2 ou 3 posições antes, diferentes da anterior. Não é observação, então não vira par.

Checagem (por dia): posições = pares + ecos + ônibus — cada ônibus-dia tem uma posição a mais que pares (a primeira não tem anterior). **Passou nos 61 dias.**

Como validamos. Primeiro um dia e 4 linhas (duas normais, duas circulares), olhando:

- a proporção de cada situação;
- a velocidade por hora (tem que cair nos picos);
- o **diagrama de Marey** — hora × metro da rota, uma linha por ônibus. Viagens aparecem como diagonais; espera no terminal, como trechos horizontais nas pontas; o `recuo` apareceu como trajetórias inteiras descendo por ~30 min — o ônibus já está voltando e a API ainda manda o sentido antigo.

Uma regra foi ajustada no teste: na 1ª versão, `terminal` era só "perto da ponta"; 3% desses pares eram ônibus **saindo ou chegando** (andando > 50 m), que passaram a contar como movimento.

Formatos v1 (2 min) e v2 (1 min) dão a mesma velocidade: 14,7 e 14,6 km/h. Dá para usar os 61 dias juntos.

2.2 `silver.gtfs_trechos` — os trechos entre paradas

Para medir velocidade por trecho, cada parada do GTFS precisa estar na mesma régua das posições (metro ao longo do traçado). As paradas de cada viagem são projetadas no traçado **em ordem** — cada uma adiante da anterior.

O problema que custou três versões: linhas que vão e voltam pela mesma avenida. A parada do início da ida também fica a poucos metros da volta (calçada oposta), lá perto do fim da rota.

Versão	Regra	Trechos	Trechos de 0 m	Erro mediano da parada
1ª	menor erro entre as passagens adiante	170.291	**23.479**	4,8 m
2ª	a passagem boa mais próxima adiante	192.949	821	10,0 m
3ª	**menor erro entre as passagens que começam até 1,5 km adiante**	**193.020**	**750**	**4,7 m**

Na 1ª, a parada pulava para a volta e todas as seguintes ficavam presas lá (uma linha de 146 paradas ficou com 10 trechos). Na 2ª, a parada era puxada para segmentos vizinhos piores. A 3ª usa o fato de que paradas ficam a ~300 m umas das outras. Cada versão foi testada localmente com traçados sintéticos antes de ir para o Databricks, e há uma checagem que falha se os trechos de 0 m passarem de 0,5%.

2.3 `silver.velocidade_trecho` — velocidade por trecho a cada 10 min

O problema: quase nenhum trecho tem uma posição dentro dele (um par anda mais que um trecho).

A solução: rateio. Cada par `ok` reparte distância e tempo entre os trechos que atravessou, supondo velocidade constante dentro do par:

```
par: metro 12.060 → 12.330 (270 m) em 45 s; parada no metro 12.150
   trecho anterior: 90 m em 15 s      trecho seguinte: 180 m em 30 s
```

Cada pedaço vai para a **janela de 10 min** do seu instante médio (casa com os pluviômetros). Em cada trecho × janela:

- **velocidade = soma das distâncias ÷ soma dos tempos** (um par curto e lento não pesa igual a um longo);
- ônibus parado conta **tempo** com distância 0;
- `n_pares` e `n_onibus` dizem quanto dado sustenta cada velocidade.

Checagens: nenhuma velocidade acima de 80 km/h; o tempo repartido não passa do tempo dos pares; **~99,6% do tempo e 99,8% da distância dos pares ficaram dentro dos trechos** (o resto é antes da 1ª parada ou depois da última). O rateio foi testado com cinco pares sintéticos (cruzando parada, parado, recuo pequeno, saindo da última parada, fora do traçado) — e esse teste pegou um erro de código antes de rodar.

Primeiro resultado com cara de resultado: o mapa de calor trecho × hora de uma linha mostra faixas lentas nos picos (a linha inteira) e colunas lentas fixas — trechos sempre lentos, inclusive de madrugada, como pontos de regulagem. Por isso a gold vai medir o **desvio de cada trecho em relação ao seu normal**, e não a velocidade bruta.

2.4 A queda de velocidade na madrugada: `parado_longo`

O sintoma: no dashboard, a velocidade média caía de madrugada (12,5 km/h às 2h), quando deveria ser a mais alta do dia. E o meio do dia parecia tão lento quanto o pico.

A investigação (13–17/10), testando três hipóteses — ônibus parados fora do terminal, composição de linhas diferente, defeito de dado:

- às 2h, **53% do tempo** dos pares `ok` era de ônibus **parado**; quem estava andando fazia **27 km/h** (o mais rápido do dia);
- de madrugada, 62% do tempo parado estava em paradas de **20 min ou mais**: 2/3 no **meio da rota** (~3 km das pontas — garagens e pontos de regulagem) e o resto a ~400 m das pontas (área de espera além dos 200 m da regra `terminal`).

O efeito de excluir paradas ≥ 20 min:

Hora	Antes	Depois
2h	12,5	**21,7**
11h	13,7	**14,9**
18h (pico)	12,1	12,6

A regra: um par que seria `ok`, dentro de uma sequência de pares parados (anda < 10 m) do mesmo ônibus que soma **20 min ou mais**, vira `parado_longo` — sai da velocidade, mas fica na tabela.

Por que 20 e não 10 min: ônibus em trânsito quase nunca fica 20 min sem andar nada, mas um **alagamento** pode prender um ônibus por 10–20 min — e isso é o que o projeto quer medir. Como a situação fica na tabela, a gold pode testar a sensibilidade (com e sem).

Curva final, dia útil (km/h): 23,0 (0h) · 22,1 (2h) · 15,6 (6h) · 14,0 (7h) · 14,9 (10h) · 14,2 (13h) · 12,5 (17h) · **11,9 (18h)** · 16,0 (20h) · 20,8 (23h).

2.5 Automação: tudo no job diário

Os notebooks 03 a 06 passaram a aceitar um parâmetro `dias`: vazio = todos; lista = só esses dias; `ontem` = o dia anterior em SP. Cada um grava **só os dias processados** (`replaceWhere`), então rodar duas vezes dá o mesmo resultado (idempotência).

O job `pipeline-onibus` ficou com 6 etapas em fila, todo dia às 10h:

```
ingestao_drive → bronze_posicao → silver_limpa → silver_rota → silver_deslocamento → silver_velocidade
```

- **Parâmetros do job** (descem para todas as etapas): `dias = ontem`, `linhas =` vazio (modo produção do 05 e 06).
- **Aviso de falha** por e-mail.
- A recuperação dos 13 dias atrasados (12–24/10) rodou com “Run now with different parameters”, em 18 min.

3. Chuva

3.1 Extração e verificação

Você baixou à mão (o site bloqueia automação) os dados de **141 postos** do município, de 01/08 a 25/10, em 17 blocos + repescagem, e documentou tudo em `CHUVA_SIBH.md`. Antes de subir, conferimos cada afirmação do documento lendo os CSVs:

O que conferiu: formato, período, nenhuma duplicata, totais por posto idênticos aos da sua tabela, e a sua hipótese de que a rede `3550308xx` **reporta por evento** (seco → 1 leitura por hora; chovendo → a cada ~10 min; leituras próximas têm chuva em 90% dos casos).

O que não conferiu, e foi corrigido:

- faltava o arquivo da repescagem (`1000868`, `527`) — você acrescentou;
- **15 valores negativos** (manutenção do pluviômetro, quase todos entre 7h e 10h, depois de chuva forte) → marcados, nunca contados como chuva;
- uma leitura de 35 mm em 10 min (210 mm/h) → marcada como implausível;
- **a rede `3550308xx` é do CEMADEN** (centro nacional de alertas), não da prefeitura.

O horário é UTC — a prova. O Windows grava o horário em que cada arquivo foi salvo, em horário de Brasília. Um arquivo salvo às **18:04** termina em `21:00`; outro, salvo às **18:55**, termina em `21:40`. Se o horário fosse de Brasília, os arquivos teriam chuva de 3 horas no futuro. Em UTC, 21:00 = 18:00 — a última leitura antes do download. (O teste pelo "pico de chuva do fim da tarde" não funcionou: setembro teve chuva frontal, concentrada de madrugada.)

Coordenadas: o `stations.json` do SIBH tem os ~3.700 postos do estado; os 141 casaram um a um, todos dentro do município.

Tudo está registrado em `docs/chuva_sibh.md` e no `manifesto.csv` (linhas, postos, período e uma impressão digital de cada arquivo).

3.2 Silver: `chuva_10min` e `postos`

`silver.chuva_10min` **— grade comum de 10 min, em UTC.** Cada leitura vale para o intervalo desde a leitura anterior do mesmo posto, repartido igualmente entre os slots de 10 min:

Caso	O que acontece
leitura a 10 min da anterior	1 slot, valor inteiro
posto a cada 5 min	duas leituras no mesmo slot, somadas
leitura horária (CEMADEN seco, INMET)	6 slots; se houver chuva, marcada `f_intervalo_longo`
intervalo maior que 60 min	só o último slot recebe; os anteriores ficam faltantes
leitura negativa	slot nulo, `f_negativo`
slot sem leitura	**nulo, nunca 0** (`f_faltante`)

Checagem de conservação: a chuva total de cada posto na grade é igual à soma das leituras da bronze — exata em 139 postos; nos outros 2, uma leitura negativa caiu no mesmo slot de uma positiva e o slot inteiro ficou nulo (0,2 e 1,1 mm).

`silver.postos` **— qualidade e status** (desde 25/08, período com ônibus): completude, maior lacuna, comparação com os 5 vizinhos mais próximos e o teste do dia 12/09 (chuva forte na cidade inteira).

Rede	ok	lacunar	quebrado
DAEE (SAISP, SP Águas, Sabesp, ANA)	71	5	6
CEMADEN	23	5	27
INMET	1	1	0

Duas lições do controle de qualidade:

1. **Vizinhos quebrados contaminam a comparação.** Na 1ª versão, 5 postos normais saíram como "suspeitos" porque os vizinhos de referência eram postos do CEMADEN com leituras zeradas. Corrigido com duas passadas.
2. **Mesmo pluviômetro cadastrado duas vezes:** `1002855` e `1001855` (Córrego Itupu) estão no mesmo ponto com o mesmo total. Marcado em `duplicata_de`.

Padrão para a gold: postos `ok` e sem duplicata — **94 postos**, cobrindo a cidade sem buracos grandes.

4. Dashboard

Dashboard **“Dash Teste”** (AI/BI), com duas abas:

- **Página 1** — feita por você na interface: velocidade por hora, chuva × velocidade por dia, mapa do pico da tarde, postos de chuva.
- **Versão API** — gerada por código a partir da sua: 4 números no topo (61 dias, 468 M posições, 29.946 trechos, 94 postos), velocidade por hora sem suavização, mapa do pico em 4 faixas de cor, chuva e velocidade por dia em gráficos separados, dispersão chuva × velocidade e um texto “Como ler”.

A definição está versionada em `dashboards/` e é aplicada pela API (`PATCH /api/2.0/lakeview/dashboards/{id}`).

Como ler a dispersão chuva × velocidade (cada ponto = um dia): ela mistura coisas demais para medir o efeito. (1) O eixo é a **mediana** dos postos, então chuva forte localizada aparece como ~0 — choveu ≥ 10 mm em algum posto em 37 dos 61 dias; (2) o tipo de dia pesa mais que a chuva (domingo ~16, dia útil ~13,5); (3) a média do dia inteiro da cidade dilui uma tempestade de uma hora. Mesmo assim, dentro dos dias úteis o efeito aparece:

Dia útil	Dias	Velocidade
seco (nenhum posto ≥ 1 mm)	3	14,4 km/h
intermediário	28	13,8 km/h
chuvoso (mediana ≥ 5 mm)	11	13,6 km/h

5. Números finais

Tabela	Linhas	Período
`bronze.posicao`	474,6 M	25/08 → 24/10 (61 dias)
`silver.posicao_limpa` / `posicao_rota`	468,4 M	idem
`silver.deslocamento`	461,3 M	idem
`silver.velocidade_trecho`	280,1 M	idem
`silver.gtfs_trechos`	193.020	versões do GTFS de 26/08 e 12/10
`bronze.chuva_sibh`	1,12 M	01/08 → 25/10
`silver.chuva_10min`	1,72 M	idem
`silver.postos`	141	—

6. O que aprendemos de método e de Databricks

Método

- **Checagem com fonte independente.** Toda tabela confere sua contagem contra a anterior (bronze = limpa + quarentena; posições = pares + ecos + ônibus; chuva conservada). Foi assim que apareceram a projeção errada das paradas e os postos contaminados.
- **Testar localmente antes de rodar.** O rateio e a projeção das paradas foram testados com dados sintéticos e pegaram erros antes de gastar processamento.
- **Classificar, não apagar.** Situações e flags em vez de linhas apagadas: dá para mudar a regra depois e medir a sensibilidade.
- **Uma linha e um dia primeiro**, olhando gráficos (Marey, mapa de calor), antes de escalar.

Databricks

- **Widgets e parâmetros de job:** o mesmo notebook roda em teste (um dia, poucas linhas, só mostra) e em produção (grava); os parâmetros do job descem para todas as etapas.
- `replaceWhere`**:** grava só os dias processados — idempotência por partição.
- `applyInPandas` **/** `cogroup`**:** lógica em pandas/numpy aplicada em paralelo por grupo (traçado × dia, posto).
- **Delta + metadados:** `overwriteSchema` apaga os comentários das colunas, então cada notebook reaplica as descrições a partir do `metadados`.
- **AI/BI Dashboards e sua API:** definição em JSON, versionável no Git.

7. Próximos passos

7.1 Gold — responder aos KRs

Passo	O que faz	KR
1. Trecho → chuva	cada trecho recebe os postos mais próximos (o mais próximo, ou média ponderada pela distância dos 3 mais próximos num raio de ~5 km), com a distância registrada	base
2. Variáveis de chuva	mm nos 10, 30 e 60 min anteriores e acumulado de 3 h, por trecho × janela	base
3. Linha de base	velocidade normal de cada trecho × hora × tipo de dia, **em períodos secos**	KR1
4. Desvio	queda da velocidade com chuva em relação à linha de base, com intervalo de confiança	KR1
5. Eventos de chuva	início, fim, total e pico por posto; para cada janela, minutos desde o fim da chuva. Horas logo depois da chuva **não** contam como secas	KR2
6. Ranking	top-10 trechos mais afetados, com estabilidade por bootstrap por dia; juntar linhas pelo **trecho físico**	KR3
7. Infraestrutura	associar a queda ao tipo de via (classificação viária da CET) e ao histórico de alagamento (GeoSampa) — dados ainda a baixar	KR4
8. Previsão (aspiracional)	degradação na hora seguinte a partir da chuva observada	KR5

Testes de sensibilidade já previstos: com e sem `parado_longo`; postos `ok` vs. `ok` + `lacunar`; limiar de "chuva" (o da OMM para chuva moderada é 2,5 mm **por hora**).

7.2 Pendências técnicas

- **GTFS novo:** o erro lateral cresce com o tempo; baixar uma versão atual (download público).
- **Recuperar o `recuo`:** as viagens com sentido atrasado da API (1,7% dos pares) são reais, só estão no traçado errado — dá para reprojetá-las no traçado do outro sentido.
- **Chuva:** novos downloads do SIBH quando houver dados novos (rodar o notebook 07 de novo).
- **KMZ e CET no Databricks:** só para validação da velocidade (o notebook 00 já aceita a pasta do KMZ).
- **Bundle (DAB):** transformar o `pipeline-onibus` em `databricks.yml` no repositório.
- **VM:** conferir o saldo da DigitalOcean (a recarga estava prevista para ~15/10).

- **Pedido LAI à CET** (histórico de lentidão), texto pronto no `CLAUDE.md`.

7.3 Para decidir com o Rafael

1. **Prazo:** ~30/10, flexível até novembro — confirmar a data final, porque a gold é a etapa mais longa.
2. **Definição de "chovendo"** para a análise e o limiar dos eventos.
3. O **[N] como regra** (mínimo de observações por trecho) e o critério de **ranking estável** do KR3.
4. Validar a regra `parado_longo` (20 min) e o desenho da linha de base.